FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Re

Abstract

Transformers are slow and memory-intensive on long sequences, as the time and memory complexity of self-attention are quadratic in sequence length. Approximate attention methods trade off model quality to reduce compute, but rarely achieve wall-clock speedups due to memory access overhead. We propose FlashAttention, an IO-aware exact attention algorithm that uses tiling to reduce the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM. FlashAttention requires O(N^2) FLOPs like standard attention but reduces memory IO from quadratic to linear in sequence length, yielding 2-4x wall-clock speedups on A100 GPUs. FlashAttention enables training Transformers with longer context, improving perplexity and enabling 64K context windows.

1. Introduction

On modern GPUs, compute speed has outpaced memory bandwidth. Most operations in standard attention (softmax, dropout, masking) are memory-bound. Standard attention materializes intermediate N x N attention matrices in HBM, causing frequent IO bottlenecks.

2. Tiling and Online Softmax
FlashAttention splits inputs into blocks of size B_r and B_c that fit entirely within fast SRAM (192 KB per streaming multiprocessor on A100). Using the online softmax trick, FlashAttention computes scaling factors incrementally without ever storing the full N x N attention matrix to HBM. During the backward pass, attention matrices are recomputed on-the-fly from SRAM rather than cached in HBM, reducing memory footprint by 10-20x.

3. Findings
On GPT-2 training with 1K context length, FlashAttention yielded a 3.1x end-to-end training speedup over HuggingFace PyTorch implementations. On long-sequence benchmarks (sequence length 4K), FlashAttention achieved a 4.2x speedup and reduced peak memory consumption by 85%. 4. Limitations
FlashAttention requires custom CUDA kernel compilation and tailored block sizing for specific GPU microarchitectures (e.g. Ampere, Hopper).